OPERATING SYSTEM KERNEL DESIGN PROJECT

Team Members:
Tenali Likhitha (241157)
B.Eepsitha (241111)
N.Hashitha (241140)

Course: Operating Systems
Department: Computer Science and Engineering

Abstract

This project focuses on designing and implementing a simple operating system kernel from scratch. The main objective of this project is to understand how an operating system works at a low level, including the booting process and interaction with hardware. The system developed successfully boots and displays the message "HELLO WORLD" using direct memory access. This project helped us gain knowledge in system-level programming, memory handling, and collaborative development using GitHub.

Introduction

An Operating System (OS) acts as an interface between the user and computer hardware. It manages system resources such as CPU, memory, and input/output devices. Modern operating systems are complex, making it difficult for beginners to understand their internal working.

This project simplifies the concept by developing a minimal OS kernel that directly interacts with hardware and displays output on the screen.

Problem Statement

Modern operating systems are complex and difficult to understand. There is a need to develop a simple system that demonstrates how an operating system boots and interacts with hardware at a low level.

Objective

- To design a simple operating system kernel
- To understand the booting process
- To display output using direct memory access
- To learn low-level programming using C and Assembly
- To execute the OS using an emulator

Tools Used

- C Programming Language
- Assembly Language (NASM)
- GCC Compiler
- QEMU Emulator
- Git and GitHub
- Ubuntu Linux

System Design

The system consists of the following flow:

- BIOS → Bootloader → Output Display

The bootloader loads and executes the program. The system then writes directly to video memory to display output on the screen.

Implementation

The project consists of the following files:

- **boot.asm** → Handles booting and initial execution
- **kernel.c** → Displays output using video memory
- **linker.ld** → Defines memory layout
- **Makefile** → Compiles and runs the OS

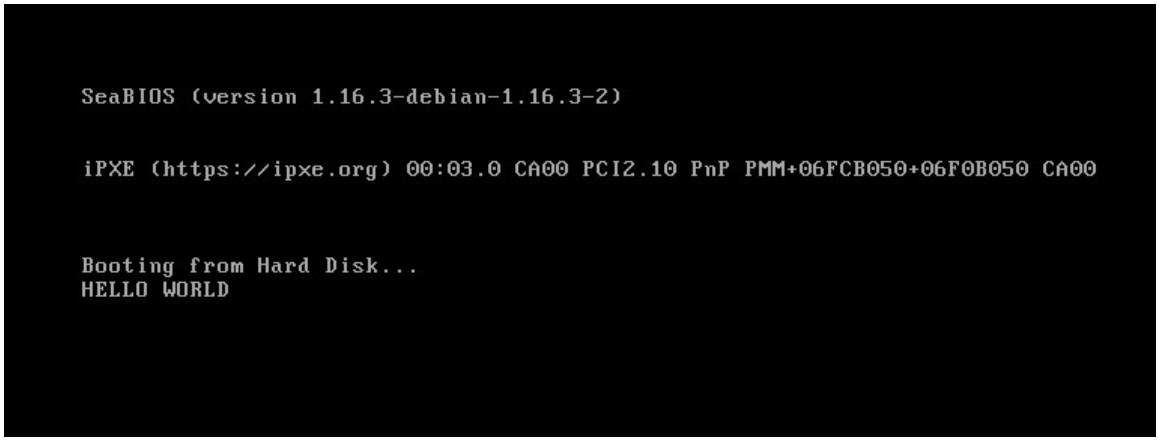
Code Snippet:

```
void kernel_main() {
        char *video_memory = (char*) 0xb8000;

        char *msg = "HELLO WORLD";

        for (int i = 0; msg[i] != '\0'; i++) {
            video_memory[i * 2] = msg[i];
            video_memory[i * 2 + 1] = 0x07;
        }
}
```

Output

The system successfully displays the message:

- HELLO WORLD



Challenges

- Errors during compilation and linking
- Understanding pointer and memory concepts
- Resolving Git conflicts

Conclusion

This project helped us understand the basic working of an operating system at a low level. We successfully developed a simple OS kernel that interacts directly with hardware and displays output. The project improved our understanding of system-level programming, debugging, and teamwork.

References

- OSDev Tutorials
- GitHub Documentation
- Online learning resources
- Course materials